

578 HW 6

Nathaniel Pulsone

2025-11-22

Question 1 (a) Below is the full Gibbs sampling algorithm

```
# load data
y1 = read.table("HW 6 files/school1.dat.txt")[,1]
y2 = read.table("HW 6 files/school2.dat.txt")[,1]
y3 = read.table("HW 6 files/school3.dat.txt")[,1]
y4 = read.table("HW 6 files/school4.dat.txt")[,1]
y5 = read.table("HW 6 files/school5.dat.txt")[,1]
y6 = read.table("HW 6 files/school6.dat.txt")[,1]
y7 = read.table("HW 6 files/school7.dat.txt")[,1]
y8 = read.table("HW 6 files/school8.dat.txt")[,1]
ylist = list("1" = y1,
             "2" = y2,
             "3" = y3,
             "4" = y4,
             "5" = y5,
             "6" = y6,
             "7" = y7,
             "8" = y8)

# Initialize prior parameters
mu_0 = 7
gam2_0 = 5
tao2_0 = 10
eta_0 = 2
sig2_0 = 15
nu_0 = 2

# Initialize sample stats and hyperparameters

m = 8
# sample size vector
n_vec = c(length(y1),
           length(y2),
           length(y3),
           length(y4),
           length(y5),
           length(y6),
           length(y7),
           length(y8))
# sample mean vector
ybar_vec = c(mean(y1),
```

```

    mean(y2),
    mean(y3),
    mean(y4),
    mean(y5),
    mean(y6),
    mean(y7),
    mean(y8))

# Initialize functions for drawing full conditionals

draw_mu = function(theta_vec, tao2){
  denom = (m/tao2 + 1/gam2_0)
  post_mean = (m*mean(theta_vec)/tao2 + mu_0/gam2_0)/denom
  post_sd = sqrt(1/denom)

  return(rnorm(1, post_mean, post_sd))
}

draw_tao2 = function(theta_vec, mu){
  post_shape = (eta_0+m)/2
  post_rate = (eta_0*tao2_0 + sum((theta_vec-mu)^2))/2

  return(1/rgamma(1, post_shape, post_rate))
}

draw_theta_vec = function(sig2, mu, tao2){
  theta_vec = c()
  for(j in 1:m){
    denom = n_vec[j]/sig2 + 1/tao2
    post_mean = (n_vec[j]*ybar_vec[j]/sig2 + mu/tao2)/denom
    post_sd = sqrt(1/denom)
    theta_vec = c(theta_vec, rnorm(1, post_mean, post_sd))
  }
  return(theta_vec)
}

draw_sig2 = function(theta_vec){
  ssr = 0

  for(j in 1:m){
    yj = as.numeric(ylist[[j]])
    ssr = ssr + sum((yj-theta_vec[j])^2)
  }
  post_shape = 0.5 * (nu_0 + sum(n_vec))
  post_rate = 0.5 * (nu_0*sig2_0 + ssr)
  sig2 = 1/rgamma(1, post_shape, post_rate)
  return(sig2)
}

# Initialize Gibbs sampler chains
# theta samples will be matrices with m cols
theta_samples = rbind(rnorm(m, mu_0, tao2_0))

```

```

sig2_samples = 1/rgamma(1, nu_0/2, nu_0/2 * sig2_0)
mu_samples = c(mu_0)
tao2_samples = c(tao2_0)

# Run Gibbs Sampler

iters = 1000

for(s in 1:(iters-1)){
  # draw mu
  mu_s = draw_mu(theta_samples[s,], tao2_samples[s])
  mu_samples = c(mu_samples, mu_s)

  # draw tao
  tao2_s = draw_tao2(theta_samples[s,], mu_s)
  tao2_samples = c(tao2_samples, tao2_s)

  # draw sig2_j's
  sig2_s = draw_sig2(theta_samples[s,])
  sig2_samples = c(sig2_samples, sig2_s)

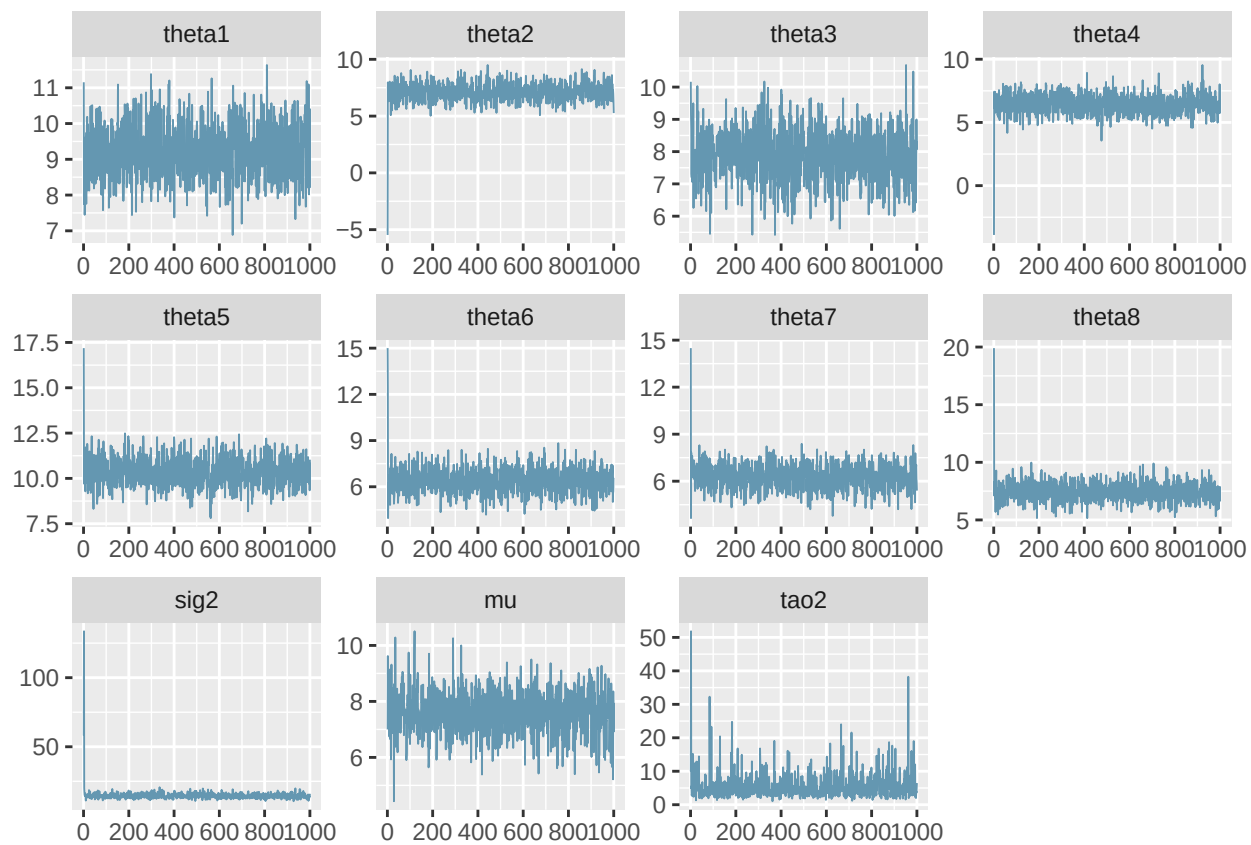
  # draw theta_j's
  theta_vec_s = draw_theta_vec(sig2_s, mu_s, tao2_s)
  theta_samples = rbind(theta_samples, theta_vec_s)
}

sim = matrix(cbind(theta_samples, sig2_samples, mu_samples, tao2_samples), ncol = 11)
colnames(sim) = c("theta1",
                  "theta2",
                  "theta3",
                  "theta4",
                  "theta5",
                  "theta6",
                  "theta7",
                  "theta8",
                  "sig2",
                  "mu",
                  "tao2")

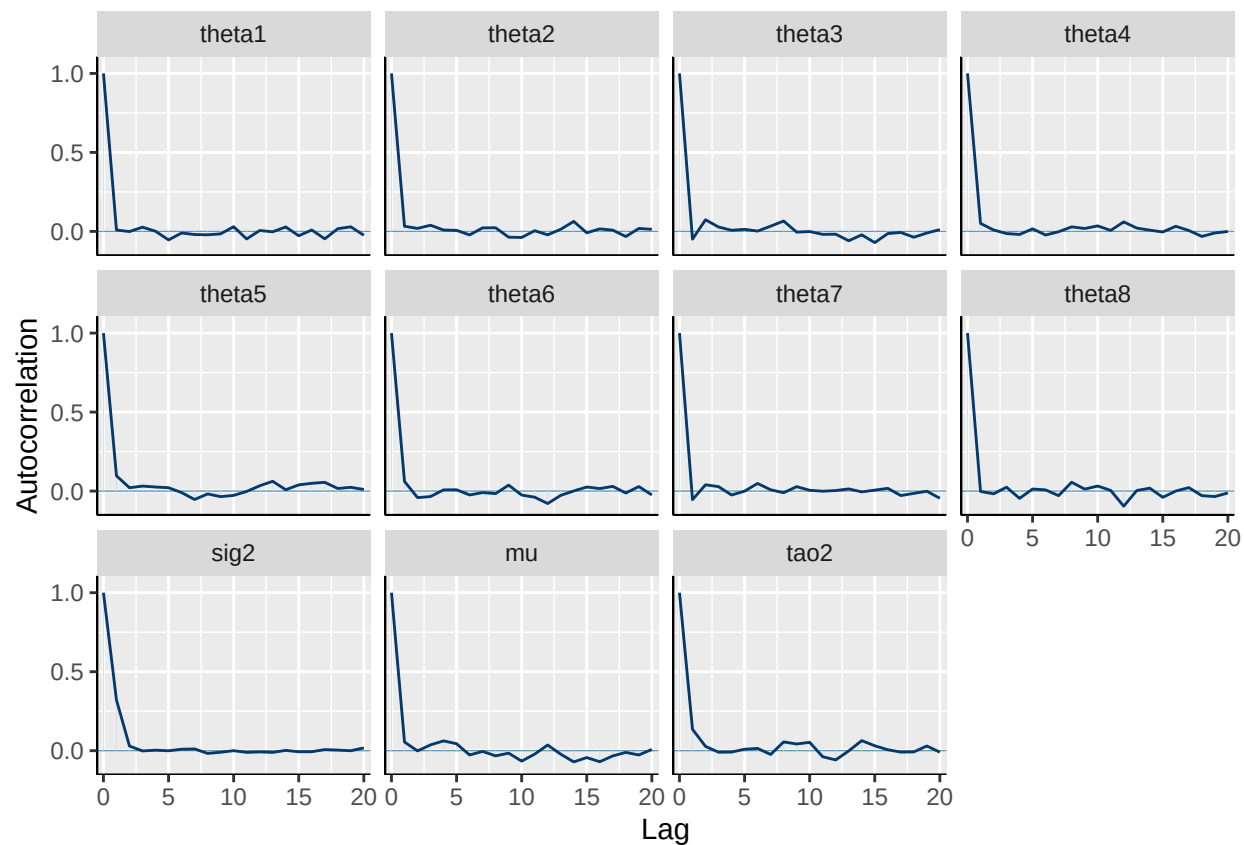
```

To examine whether the markov chain has converge, we can look at the trace plot and autocovariance function. Based on the plots below, there are neither global nor local trends in any trace plot, and the acf decays rapidly, it is therefore safe to assume that the markov chain converged.

```
mcmc_trace(sim)
```



```
mcmc_acf(sim)
```



```
rm(sim)
```

Question 1(b) below are the posterior means sigma, mu and tao respectively:

```
mean(sig2_samples)
```

```
## [1] 14.66131
```

```
mean(mu_samples)
```

```
## [1] 7.572705
```

```
mean(tao2_samples)
```

```
## [1] 5.404854
```

Below are the confidence intervals for sigma, mu and tao respectively:

```
quantile(sig2_samples, c(0.025, 0.975))
```

```
##      2.5%      97.5%
```

```
## 11.75664 18.10819
```

```
quantile(mu_samples, c(0.025, 0.975))
```

```
##      2.5%      97.5%
## 6.017137 8.973507
```

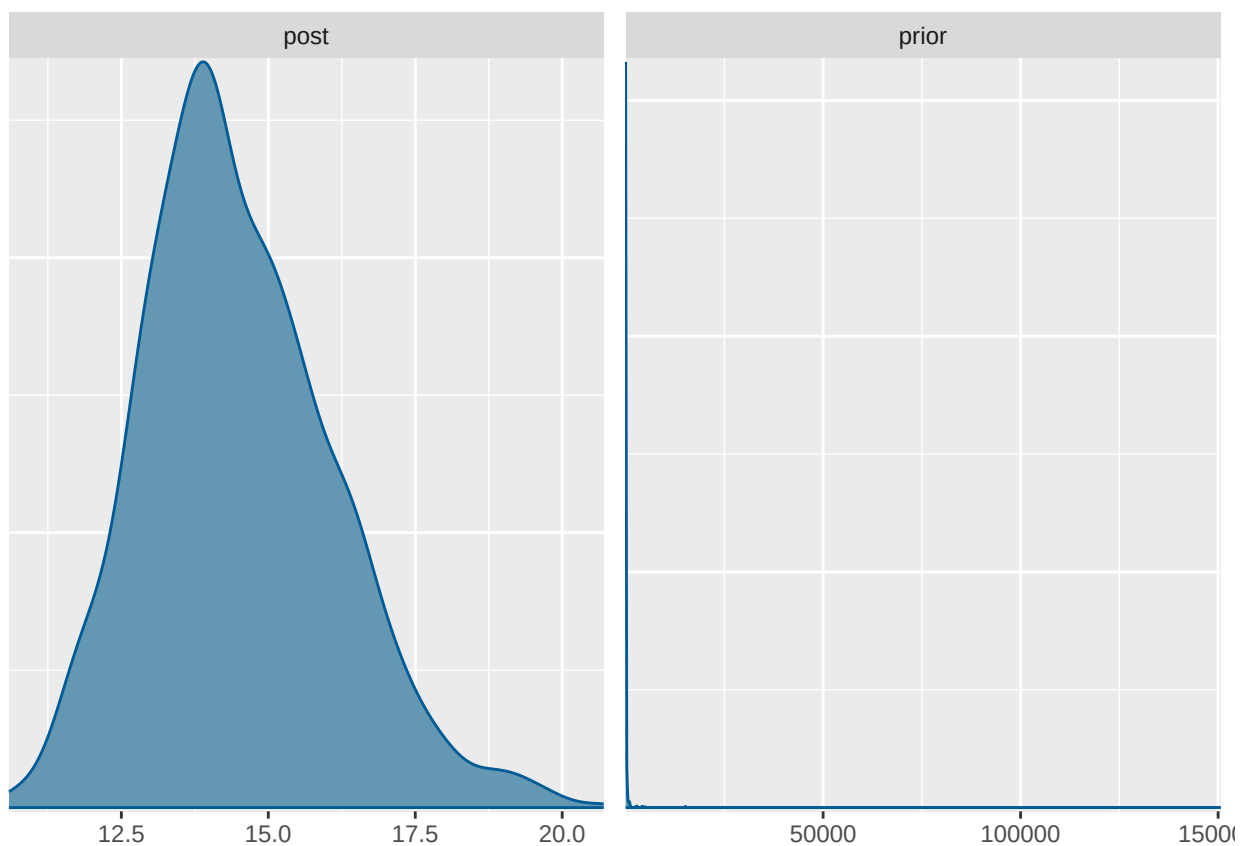
```
quantile(tao2_samples, c(0.025, 0.975))
```

```
##      2.5%      97.5%
## 1.869846 15.199019
```

Below is an overlay of the prior and posterior density plots for sigma, we see that the posterior is much less skewed, and more concentrated around 15. The data reduces uncertainty about the value of the parameter, suggesting that it is highly likely to be somewhere between 12.5 and 17.5.

```
sig2_prior_samples = 1/(rgamma(900, nu_0/2, nu_0/2 * sig2_0))
sim = matrix(cbind(sig2_samples[101:1000], sig2_prior_samples), ncol = 2)

colnames(sim) = c("post", "prior")
mcmc_dens(sim)
```



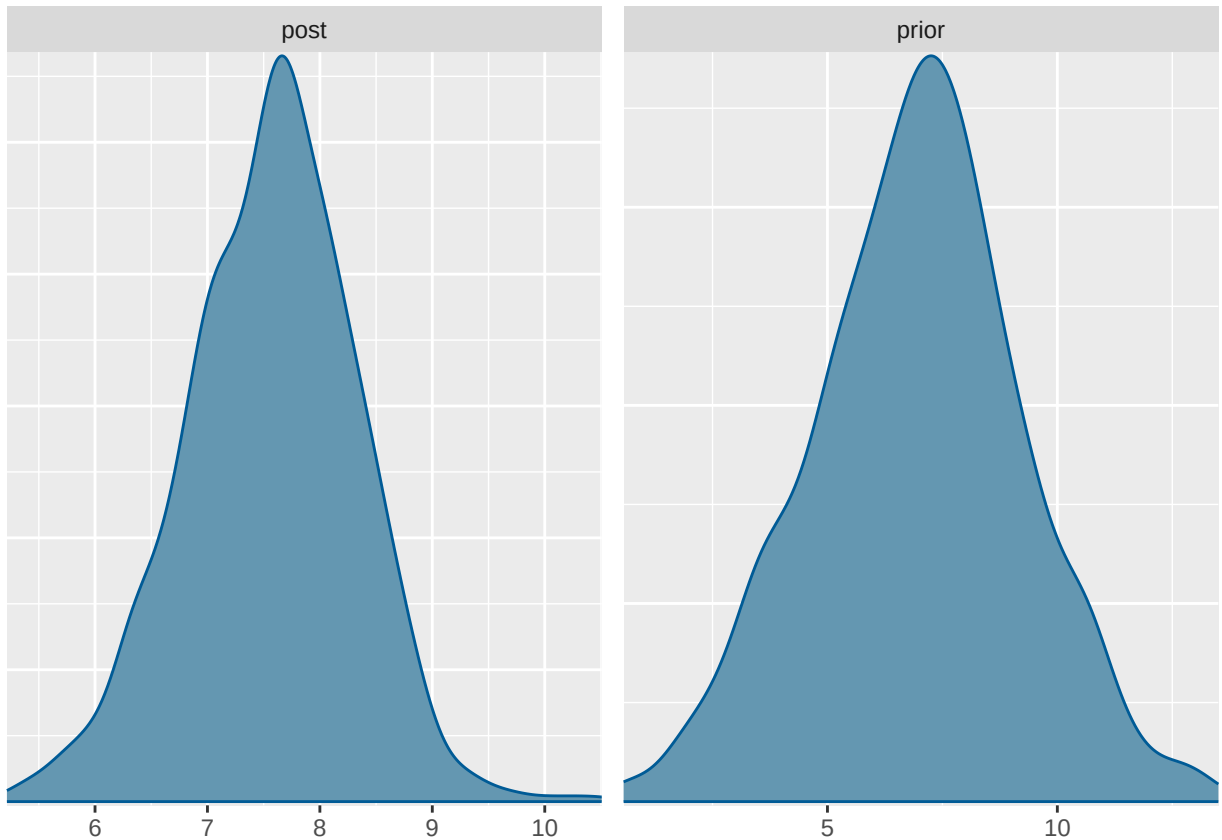
Below is an overlay of the prior and posterior density plots for mu, we see that the posterior is shifted right a bit and more concentrated. The data reduces uncertainty about the value of the parameter, suggesting that it is higher than the prior suggests, and somewhere between 6 and 9.

```

mu_prior_samples = rnorm(900, mu_0, sqrt(gam2_0))
sim = matrix(cbind(mu_samples[101:1000], mu_prior_samples), ncol = 2)

colnames(sim) = c("post", "prior")
mcmc_dens(sim)

```



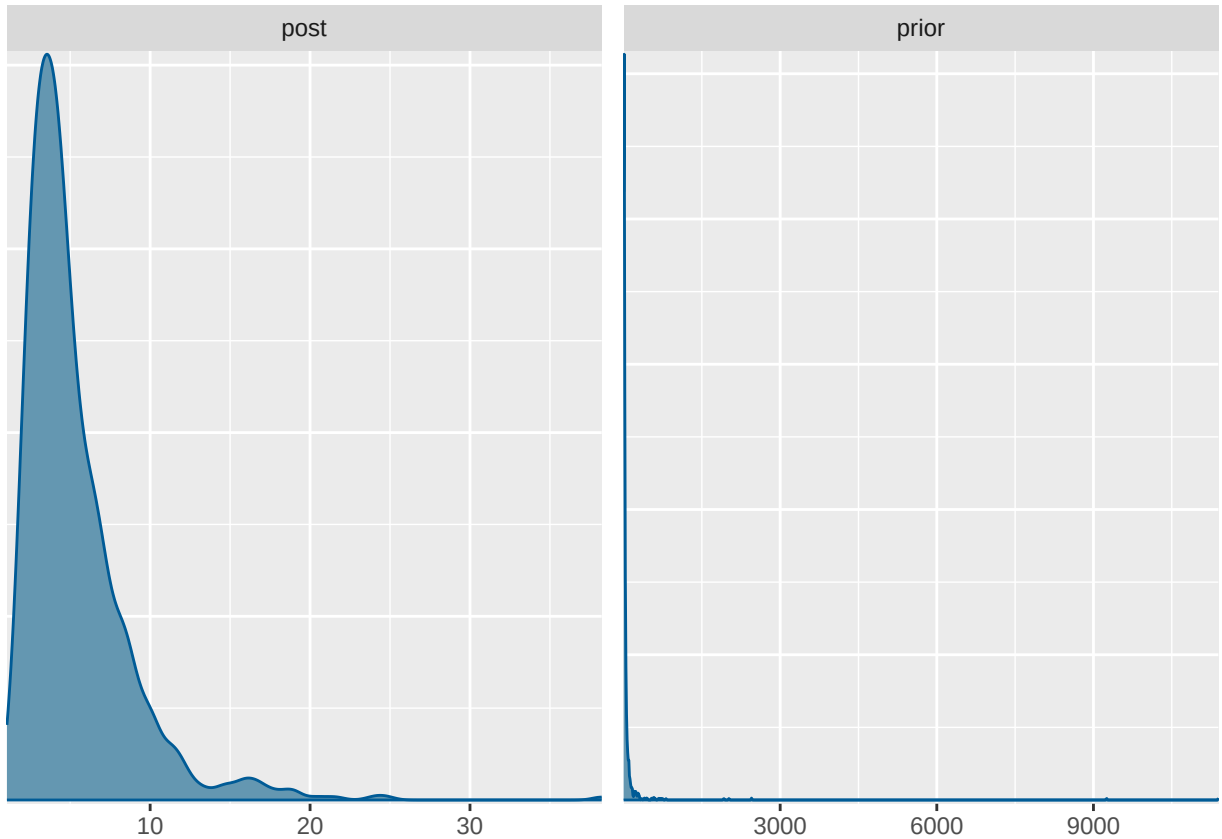
Below is an overlay of the prior and posterior density plots for τ , we see that the posterior is much less skewed and more concentrated. The data reduces uncertainty about the value of the parameter, and is highly likely to be somewhere between 0 and 10.

```

tau_prior_samples = 1/rgamma(900, eta_0/2, eta_0/2 * tau2_0)
sim = matrix(cbind(tau2_samples[101:1000], tau_prior_samples), ncol = 2)

colnames(sim) = c("post", "prior")
mcmc_dens(sim)

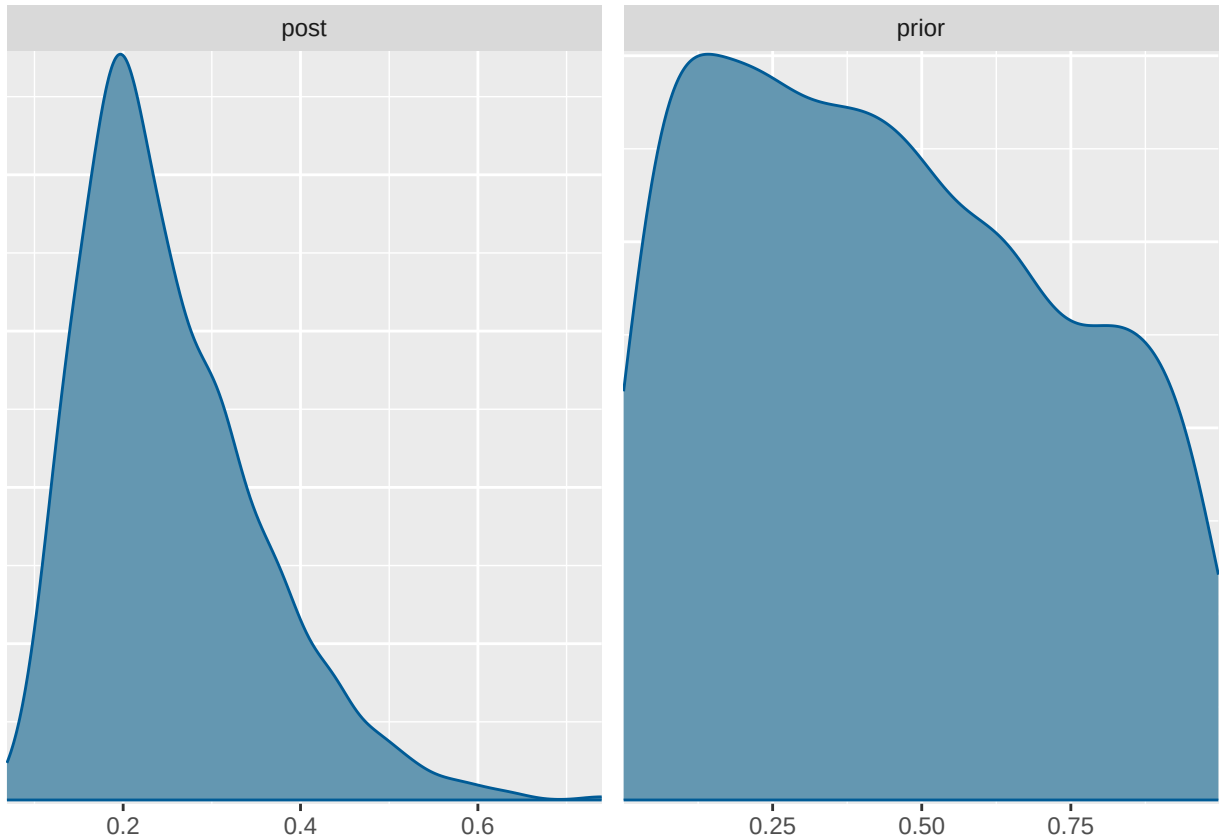
```



Question 1(c) Below is a comparison of the prior and posterior distributions of R . We see that R is lower and more concentrated around 0.2, which suggests that between-school variation is low.

```
R = tao2_samples[101:1000]/(tao2_samples[101:1000] + sig2_samples[101:1000])
Rprior = tao_prior_samples/(tao_prior_samples + sig2_prior_samples)
sim = matrix(cbind(R, Rprior), ncol = 2)

colnames(sim) = c("post", "prior")
mcmc_dens(sim)
```



Question 1(d) Below is the computed probability that $\theta_7 < \theta_3$ estimated from the Gibbs Sampler. The probability is estimated to be 0.929, which provides strong suggestion that $\theta_7 < \theta_3$.

```
mean(theta_samples[,7] < theta_samples[,3])
```

```
## [1] 0.916
```

Below is the computed probability that θ_7 is less than all other θ s, estimated from the Gibbs Sampler. The probability is estimated to be 0.33, which suggests there is not much evidence to support this conclusion.

```
a = 1 * (theta_samples[,7] < theta_samples[,1]) * (theta_samples[,7] < theta_samples[,2]) * (theta_samp
mean(a)
```

```
## [1] 0.344
```

Question 1(e) Below is the plot of posterior means against sample means, with a line $y=x$ drawn through the plot. We see a linear relationship between the two, where it appears that the slope is slightly less than 1, meaning that there is a minor shrinkage effect which otherwise would suggest an identical relationship.

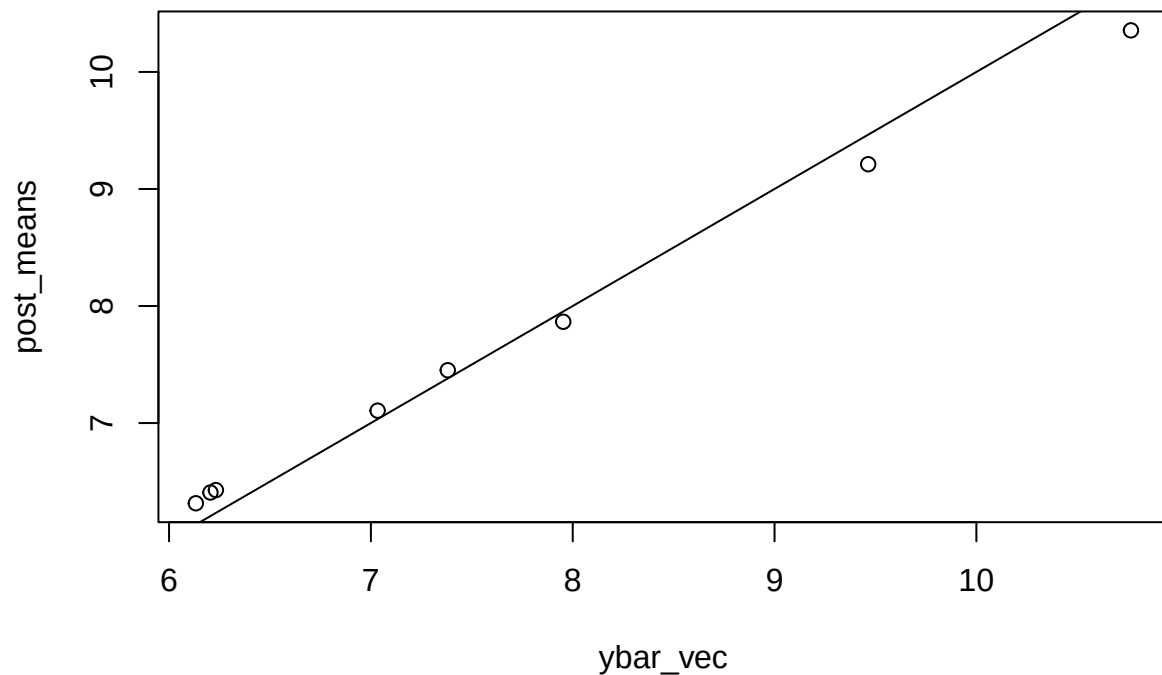
```
post_means = c(mean(theta_samples[,1]),
               mean(theta_samples[,2]),
               mean(theta_samples[,3]),
               mean(theta_samples[,4]),
               mean(theta_samples[,5]),
```

```

mean(theta_samples[,6]),
mean(theta_samples[,7]),
mean(theta_samples[,8]))

plot(ybar_vec, post_means)
abline(0, 1)

```



```
mean(c(y1, y2, y3, y4, y5, y6, y7, y8))
```

```
## [1] 7.691278
```

```
mean(mu_samples)
```

```
## [1] 7.572705
```

We see that the global sample mean is 7.69, while the posterior mean of μ is 7.52. The two are very close in value meaning the posterior is likely weighted heavily in favor of the data, however there is a small shrinkage effect on the posterior mean value of μ due to the prior.

Question 2 (a)

```

dat = read.table("HW 6 files/pdensity.dat.txt")[2:81,]
colnames(dat) = c("plot", "density", "yield")

```

```

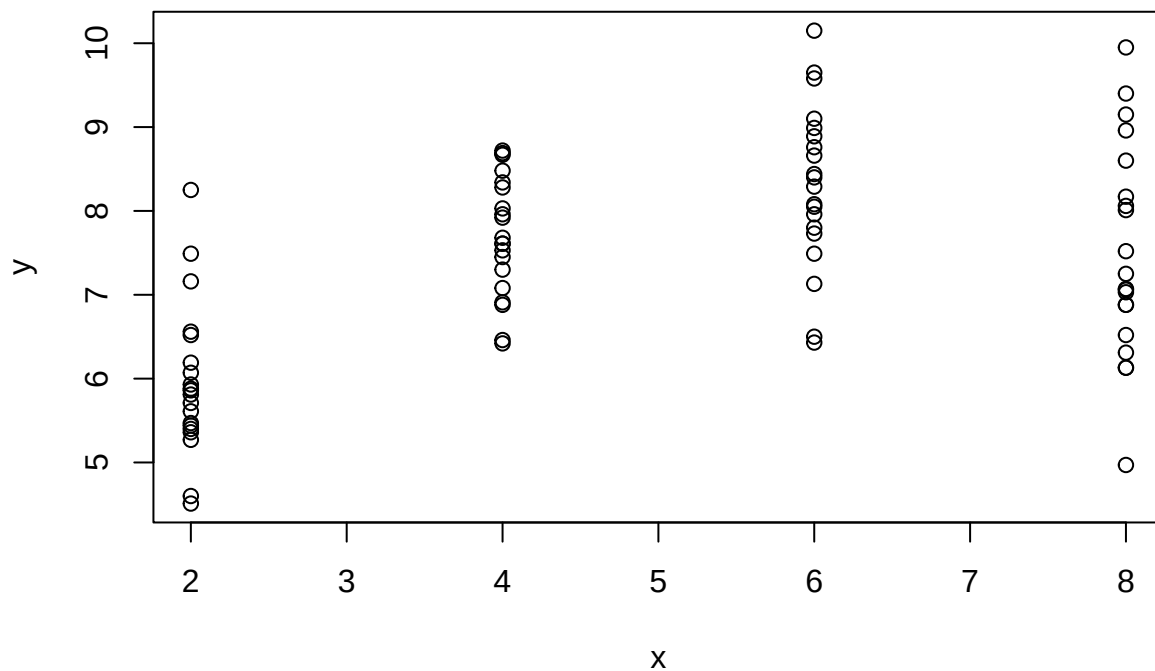
y = dat$yield
x = dat$density

ols.lm = lm(y ~ x + x^2)

coefs = coef(ols.lm)

plot(x, y)
x = as.numeric(x)
yfit = coefs[3]*x^2 + coefs[2]*x + coefs[1]
lines(x, yfit)

```



Below are ad hoc estimates for theta and Sigma (coefficient covariance matrix). Theta's estimate will be the coefficients of the linear model, and Sigma will be the psuedo inverse of the design matrix, and σ^2 will be the variance of the residuals

```

theta_hat = as.numeric(coefs)
sig2_hat = var(ols.lm$residuals)

X = cbind(1, x, x^2)

Sigma_hat = sig2_hat*solve(t(X)%*%X)
Sigma_hat_inv = t(X)%*%X / sig2_hat

theta_hat

```

```
## [1] 5.9545 1.7465 2.3495 1.5480
```

```
sig2_hat
```

```
## [1] 0.9393485
```

```
Sigma_hat
```

```
##              x
## 0.36399754 -0.15851506 0.014677320
## x -0.15851506 0.07573497 -0.007338660
## 0.01467732 -0.00733866 0.000733866
```

Question 2 (b) Below is the Gibbs Sampler algorithm:

```
y = as.numeric(y)
m = 10
Lambda_0 = diag(1, 3, 3)
mu_0 = 0
plot = as.numeric(dat$plot)
ylist = list("1" = y[which(plot==1)],
             "2" = y[which(plot==2)],
             "3" = y[which(plot==3)],
             "4" = y[which(plot==4)],
             "5" = y[which(plot==5)],
             "6" = y[which(plot==6)],
             "7" = y[which(plot==7)],
             "8" = y[which(plot==8)],
             "9" = y[which(plot==9)],
             "10" = y[which(plot==10)]
            )

xlist = list("1" = x[which(plot==1)],
             "2" = x[which(plot==2)],
             "3" = x[which(plot==3)],
             "4" = x[which(plot==4)],
             "5" = x[which(plot==5)],
             "6" = x[which(plot==6)],
             "7" = x[which(plot==7)],
             "8" = x[which(plot==8)],
             "9" = x[which(plot==9)],
             "10" = x[which(plot==10)])

# update functions
draw_betaj = function(yj, Xj, sig2, theta, Sigma_inv){
  varbj = solve(Sigma_inv + t(Xj)%*%Xj/sig2)
  ebj = varbj %*% Sigma_inv %*% theta + (t(Xj) %*% yj)/sig2
  return(mvrnorm(1, ebj, varbj))
}

draw_theta = function(betabar, Sigma_inv){
  Lambda_m = solve(Lambda_0 + m * Sigma_inv)
```

```

mu_m = Lambda_m %*% (m * Sigma_inv %*% betabar)
return(mvrnorm(1, mu_m, Lambda_m))
}

draw_Sigma_inv = function(Beta, theta){
  df = 4 + m
  S_theta = Sigma_hat_inv
  for(j in 1:m){
    betaj = Beta[j,]
    S_theta = S_theta + (betaj - theta)%*%t(betaj - theta)
  }

  return(rWishart(1, df, S_theta))
}

draw_sig2 = function(Beta){
  shape = 40.5
  ssr = 0
  for(j in 1:m){
    xj = xlist[[j]]
    Xj = cbind(1, xj, xj^2)
    ssr = ssr + sum((ylist[[j]] - Xj%*%Beta[j,])^2)
  }
  rate = (sig2_hat + ssr)/2
  return(1/rgamma(1, shape, rate))
}

iters = 700

Beta_samples = array(0, dim = c(m, 3, iters))
Beta_samples[,1] = cbind(rep(theta_hat[1], 10),
                        rep(theta_hat[2], 10),
                        rep(theta_hat[3], 10))
Sigma_inv_samples = array(0, dim = c(3, 3, iters))
Sigma_inv_samples[,1] = Sigma_hat
sig2_samples = c(sig2_hat)
theta_samples = array(0, dim = c(iters, 3))
theta_samples[1,] = theta_hat[2:4]

for(s in 1:(iters-1)){
  sig2_s = draw_sig2(Beta_samples[, ,s])
  sig2_samples = c(sig2_samples, sig2_s)

  betabar = c(mean(Beta_samples[,1,s]),
              mean(Beta_samples[,2,s]),
              mean(Beta_samples[,3,s]))

  theta_samples[s+1,] = draw_theta(betabar, Sigma_inv_samples[, ,s])

  Sigma_inv_samples[, ,s+1] = draw_Sigma_inv(Beta_samples[, ,s],
                                             theta_samples[s+1,])
}

```

```

for(j in 1:m){
  yj = ylist[[j]]
  xj = xlist[[j]]
  Xj = cbind(1, xj, xj^2)
  Beta_samples[j,,s+1] = draw_betaj(yj, Xj, sig2_s,
                                   theta_samples[s+1,],
                                   Sigma_inv_samples[, ,s+1])
}
}

```

```

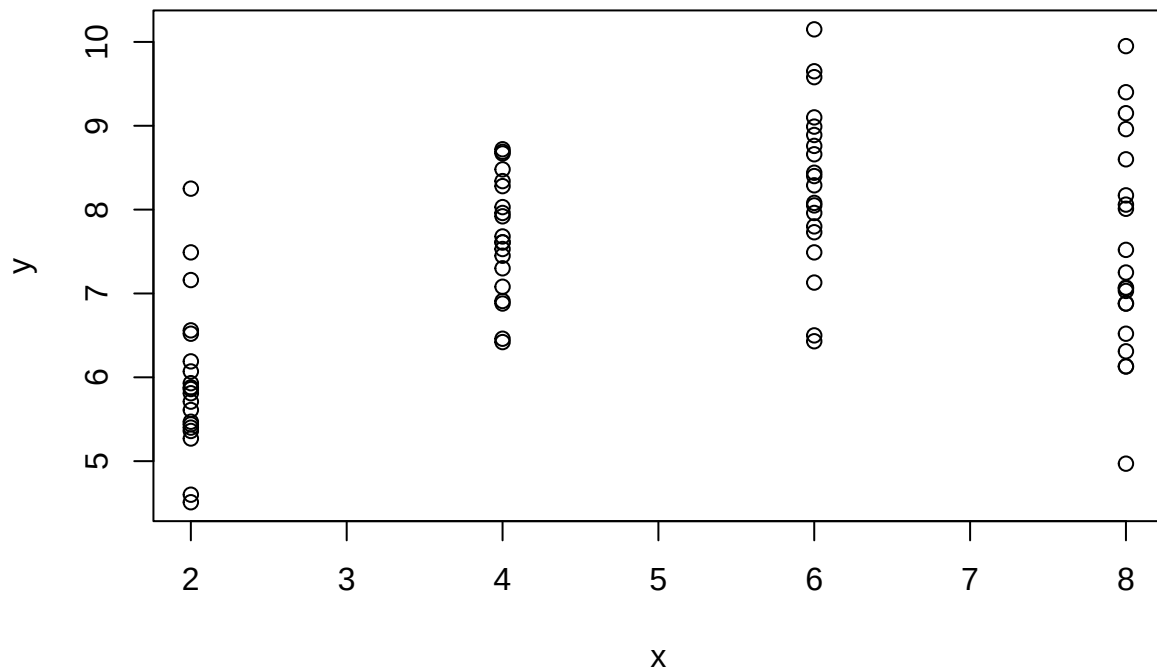
beta_0_hat = c(mean(Beta_samples[1,1,]),
               mean(Beta_samples[2,1,]),
               mean(Beta_samples[3,1,]),
               mean(Beta_samples[4,1,]),
               mean(Beta_samples[5,1,]),
               mean(Beta_samples[6,1,]),
               mean(Beta_samples[7,1,]),
               mean(Beta_samples[8,1,]),
               mean(Beta_samples[9,1,]),
               mean(Beta_samples[10,1,]))

beta_1_hat = c(mean(Beta_samples[1,2,]),
               mean(Beta_samples[2,2,]),
               mean(Beta_samples[3,2,]),
               mean(Beta_samples[4,2,]),
               mean(Beta_samples[5,2,]),
               mean(Beta_samples[6,2,]),
               mean(Beta_samples[7,2,]),
               mean(Beta_samples[8,2,]),
               mean(Beta_samples[9,2,]),
               mean(Beta_samples[10,2,]))

beta_2_hat = c(mean(Beta_samples[1,3,]),
               mean(Beta_samples[2,3,]),
               mean(Beta_samples[3,3,]),
               mean(Beta_samples[4,3,]),
               mean(Beta_samples[5,3,]),
               mean(Beta_samples[6,3,]),
               mean(Beta_samples[7,3,]),
               mean(Beta_samples[8,3,]),
               mean(Beta_samples[9,3,]),
               mean(Beta_samples[10,3,]))

plot(x, y)
for(j in 1:m){
  yfit = yfit = beta_2_hat[j]*x^2 + beta_1_hat[j]*x + beta_0_hat
}
lines(x, yfit)

```

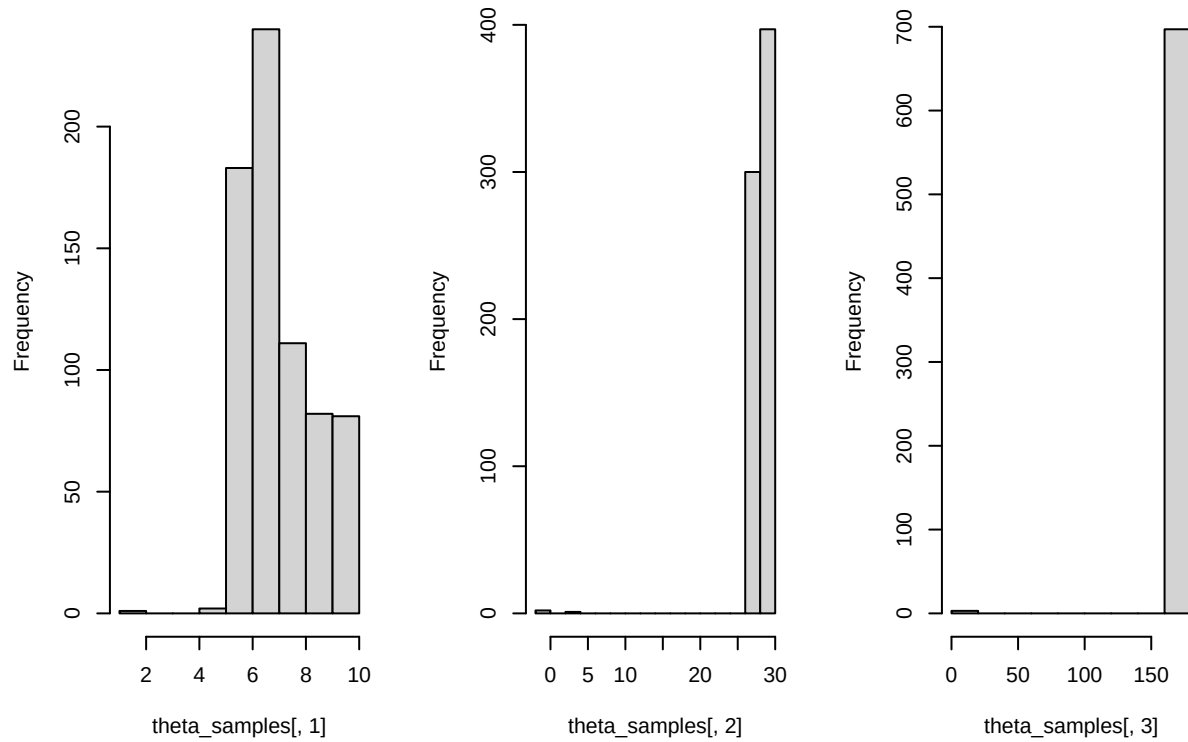


For the most part, there is not a significant difference between the posterior expectations for beta in each group, and neither do these significantly differ from the coefficients obtained in the OLS. Due to the fact that OLS was used to formulate the prior for the Gibbs sampler, it makes sense that the two methods would agree to some extent. The lack of variation between groups is suggestive that the effect in planting density across different plots is roughly the same for all plots.

Question 1(c) Below is the density plot for the elements in theta and sigma inverse

```
par(mfrow = c(1, 3))
hist(theta_samples[,1])
hist(theta_samples[,2])
hist(theta_samples[,3])
```

Histogram of theta_samples[, 1] Histogram of theta_samples[, 2] Histogram of theta_samples[, 3]

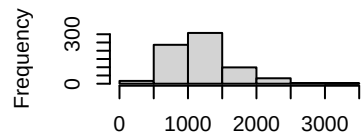


```
par(mfrow = c(3, 3)) # precision
hist(Sigma_inv_samples[1,1,])
hist(Sigma_inv_samples[1,2,])
hist(Sigma_inv_samples[1,3,])

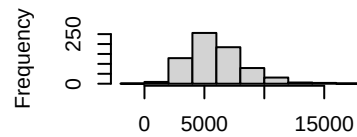
hist(Sigma_inv_samples[2,1,])
hist(Sigma_inv_samples[2,2,])
hist(Sigma_inv_samples[2,3,])

hist(Sigma_inv_samples[3,1,])
hist(Sigma_inv_samples[3,2,])
hist(Sigma_inv_samples[3,3,])
```

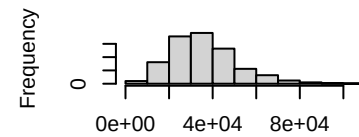
istogram of Sigma_inv_samples[istogram of Sigma_inv_samples[istogram of Sigma_inv_samples[



Sigma_inv_samples[1, 1,]

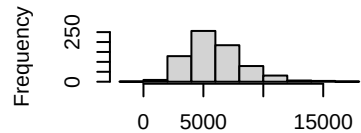


Sigma_inv_samples[1, 2,]

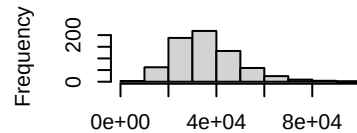


Sigma_inv_samples[1, 3,]

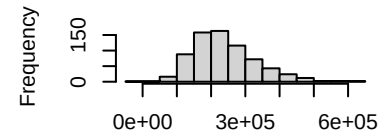
istogram of Sigma_inv_samples[istogram of Sigma_inv_samples[istogram of Sigma_inv_samples[



Sigma_inv_samples[2, 1,]

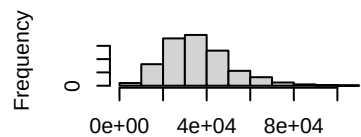


Sigma_inv_samples[2, 2,]

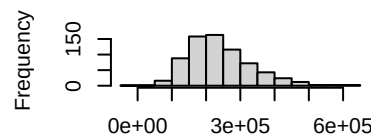


Sigma_inv_samples[2, 3,]

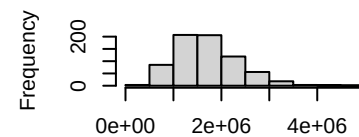
istogram of Sigma_inv_samples[istogram of Sigma_inv_samples[istogram of Sigma_inv_samples[



Sigma_inv_samples[3, 1,]



Sigma_inv_samples[3, 2,]



Sigma_inv_samples[3, 3,]

As in our earlier analysis, the distributions seem to suggest that the across group heterogeneity is quite small. First, we see that each distribution in theta has fairly low variance and is centered around small values, meaning that Beta as a result will not have too much variance across groups. Next, we see that the distribution for the elements in sigma inverse are centered around very large values, which implies high percision / low variability in beta across groups.

Question 1(d) Below, we see that yield is maximized when the density is set to 6.095. The posterior predictive distribution will be normal with variance sig2, so the interval is given as below:

```
x_max = 6.095
y_max_hat = mean(beta_0_hat) + mean(beta_1_hat)*x_max + mean(beta_2_hat)*x_max^2
c(y_max_hat - mean(sig2_samples), y_max_hat + mean(sig2_samples))
```

```
## [1] -43589618 43602609
```